

UNIVERSITY OF LEEDS

COMP1850 Programming

Week 4 Session 2

More on functions

Recap

UNIVERSITY OF LEEDS

Last session, we have looked at:

- how to define a function – with `def`

```
def add_two_numbers(num1, num2):
    return (num1 + num2)
```

- how to call a function

```
result = add_two_numbers(3, 4)
```

Today’s aims

UNIVERSITY OF LEEDS

To introduce :

- Optional arguments
- Functions that return multiple values
- Recursive functions

Optional arguments

UNIVERSITY OF LEEDS

It is possible to define a function that accepts an argument
but uses a default value if the argument is not provided.

- You have already seen examples of functions with optional arguments.
    - For example, the print() and range() functions

Defining optional arguments

UNIVERSITY OF LEEDS

We can assign default values to function parameters when we define the function

- Assignment uses '='

Eg.
```
def func( p_one, p_two=0 ) :
    ...
```

Parameter p_one is required

Parameter p_two is optional and has value 0 if not provided

Example: range()

UNIVERSITY OF LEEDS

The range() function can be used in several ways:

- range(4) -> 0,1,2,3
- range(3,6) -> 3,4,5
- range(1,8,2) -> 1,3,5,7

Most general use :

range(start, stop, step)

start and step are optional when we use the function

Defining range()

The function definition for range() :

```
def range( start=0, stop, step=1 )
```

start has a default value 0

step has a default value 1

stop is required

Example: print()

UNIVERSITY OF LEEDS

Although we use print() in a simple way to print a string to the terminal it has a complex definition with several optional arguments

See https://docs.python.org/3/library/functions.html#print

"Print *objects* to the text stream *file*, separated by *sep* and followed by *end and flush.*
end, file, and *flush*, if present, must be given as keyword arguments"

def print(**objects*, ***sep=' ', end='\n', file=None, flush=False***)

Example: print() parameters

UNIVERSITY OF LEEDS

print(**objects*, ***sep=' ', end='\n', file=None, flush=False***)

- sep separator between strings, default empty string (' ')
- end printed after the final string, default new line (\n)
- file print destination, default None is the terminal (stdout)
- flush flush the print buffer, default False

Optional print() arguments

UNIVERSITY OF LEEDS

For example,

```
for item in range(5):
    print(item, end=":")
```

prints

```
0:1:2:3:4:
```

Compare to the default behaviour

Optional arguments

UNIVERSITY OF LEEDS

- Parameters that do not have a default value are required (mandatory)
- Useful for more complex functions to specify :
  - which parameters require a value
  - named optional parameters
- Good documentation for your function is essential
  - To specify required arguments
  - To specify default behaviour for optional arguments

Optional arguments in our own functions

UNIVERSITY OF LEEDS

- The default value is assigned by using the assignment (=) operator

```
def greet(name, greeting="Hello", punctuation="!"):
    return f"{greeting}, {name}{punctuation}"


print(greet("Charlie")) # Hello, Charlie!

print(greet("Bob", "Good morning")) # Good morning, Bob!

print(greet("Alice", "Hi", "."))     #  Hi, Alice.
```

Task 1

UNIVERSITY OF LEEDS

In the task 1 folder:

- In default.py, you are asked to change the default value to your name and run the program
- In format_date.py, write a function with an optional argument, and set a default value for that argument
- In circle_area.py, complete the function to calculate area of a circle and set a default value for the optional argument Pi.
- Find out more about the calendar.month() method which accepts optional arguments with default values. Then, write calls to the method and provide different values to the optional arguments.

Functions returning multiple values

UNIVERSITY OF LEEDS

- So far, we have only looked at functions returning a single value
- However, we can define functions that returns multiple values
- Simply write a comma separated list of all the values to return.

```
def return_constants():
    PI = 3.14159
    GRAVITY = 9.81
    SPEED_OF_LIGHT = 299792458
    return PI, GRAVITY, SPEED_OF_LIGHT
```

The returned values are a tuple

UNIVERSITY OF LEEDS

- The calling code should assign these values to a tuple

```
constants = return_constants()
print( constants )
# (3.14159, 9.81, 299792458)
```

- Or, unpack the tuple to separate values

```
pi, gravity, light_speed = return_constants()

print(pi, gravity, light_speed)
# 3.14159, 9.81, 299792458
```

Task2

UNIVERSITY OF LEEDS

In the task 2 folder, you will be working in writing functions that take one or more arguments, and returning different number of values.

- In odd_even.py, complete the function to return the number of odd integers, and the number of even integers, in a list
- In quotient.py, complete the function to return the quotient and remainder of a number
- In highest_vote.py, complete the function that will return the item with the highest number of occurrences, and return the item and the count of occurrences
- In point.py, write a function point_properties that takes 4 arguments (2 points) and returns the distance and the mid point between the two points.

Recursive functions

UNIVERSITY OF LEEDS

Recursion is a method where the solution to a problem is calculated from solutions of smaller instances of the same problem.

Three golden rules:

- A recursive function must have a **base case**
- A recursive function must **call itself (recursive case)**
- The **recursive case** must approach the **base case**

A simple example: factorial(n)

UNIVERSITY OF LEEDS

factorial(n) = n x (n-1) x (n-2) x … x 3 x 2 x 1

Recursion can be used since:

factorial(n) = n x factorial(n-1)
= n x ((n-1) x factorial(n-2))
= ...

and we have the base case (which we approach)

factorial(1) = 1

Code: factorial(n) function

UNIVERSITY OF LEEDS

```
def factorial( n ):
    if n == 1:    # base case
        return 1
    return n*factorial(n-1) # recursive call reducing n

factorial(5)    # result will be 120
```

Types of recursion

The factorial(n) case is an example of **linear recursion**

- The function makes one recursive call to itself

There are other categories of recursion

- A common type is **tree recursion**
- The function makes 2 (or more) recursive calls

A further example: fibonacci(n)

UNIVERSITY OF LEEDS

Fibonacci numbers are a sequence: 0,1,1,2,3,5,8,13,21,...

Each Fibonacci number is the sum of the 2 previous numbers.

Taking the base case: Fibonacci(0) = 0, Fibonacci(1) = 1,

we can write the nth Fibonacci number recursively as :

$$\begin{aligned} \text{Fibonacci}(n) &= \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2) \\ &= (\text{Fibonacci}(n-2) + \text{Fibonacci}(n-3)) \\ &\quad + (\text{Fibonacci}(n-3) + \text{Fibonacci}(n-4)) \\ &= \ldots \end{aligned}$$

Code: fibonacci(n) function

UNIVERSITY OF LEEDS

```
def fibonacci( n ) :
    if n == 0:                                  # base case
        return 0
    elif n == 1:                               # base case
        return 1
    return fibonacci(n-1) + fibonacci(n-2) # 2 recursive calls

result = fibonacci(8)       # result will be 21
```

Why use recursive functions?

UNIVERSITY OF LEEDS

Recursive algorithms are often concise and elegant

- Implementation requires less code
- Natural fit for some data structures (trees, graphs)

We can always rewrite a recursive algorithm in a non-recursive form

- Useful for testing as recursive functions are difficult to debug

Task 3

UNIVERSITY OF LEEDS

In task3, you will write recursive functions.

In each case consider the recursive solution and the base case

- digits_sum.py: complete the function
- countdown.py: complete the function
- power.py: write the recursive function

Development task

UNIVERSITY OF LEEDS

Last week we started creating code for PGM image files

This session we can write functions for common operations

- Reading PGM files from disk
- Writing PGM files to disk
- Simple operations on PGM files

Task 4

UNIVERSITY OF LEEDS

Templates are provided to start.

Think about:

- How to store a PGM image
- How to pass a PGM image to/from a function

Some hints are provided

Summary

UNIVERSITY OF LEEDS

We have covered a lot of areas that illustrate the flexibility we have when using functions

- In the parameter list when defining functions
- In the argument list when calling a function
- In the output when returning from a function

Recursive algorithms become more important when we look at more general data structures, in Programming and in Theory modules.